Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ” Python: funcții, metode și importuri pe web”

A efectuat : st. gr.SI – 263 Morozovschii Carolina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

În acest laborator veți scrie programe Python care comunică cu un site web real și veți exersa trei concepte de bază: **importurile**, **metodele** și **funcțiile**. La final veți avea propriul modul Python, care generează un raport despre un site web.

Capitol 1: Importuri (exercițiile 1–8)

Scop: învățați diferitele moduri de a aduce cod în program. Fișier: `part1_imports.py`.

Exercițiul 1. Primul import. Scrieți `import requests` și afișați `requests.__version__`. Apoi scrieți `import urllib.request`. Într-un comentariu, explicați de ce `requests` a avut nevoie de `pip install`, iar `urllib` nu.

Exercițiul 2. Două stiluri de import. Descărcați pagina `BASE_URL` cu `requests.get()` și afișați codul de stare. Apoi rescrieți același script folosind `from requests import get`. Într-un comentariu, scrieți câte un avantaj al fiecărui stil.

Exercițiul 3. Alias-uri. Importați requests cu un alias: `import requests as rq`. Descărcați pagina principală cu `rq.get()`. Într-un comentariu, spuneți când un alias face codul mai ușor de citit și când îl face mai greu de citit.

Exercițiul 4. Doar biblioteca standard. Descărcați pagina principală folosind doar `urllib.request.urlopen()`. Afișați atributul `.status` al răspunsului și primele 200 de caractere din corp.

> **Indiciu:** `.read()` returnează octeți (bytes). Folosiți `.decode("utf-8")` ca să îi transformați într-un șir de caractere.

Exercițiul 5. Priviți în interiorul unui modul. Rulați `print(dir(requests))`. Alegeți trei nume din rezultat și scrieți, într-un comentariu, dacă fiecare este o funcție, o clasă sau un modul.

Exercițiul 6. Citiți documentația. Rulați `help(requests.get)`. Găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

Exercițiul 7. Cronometrarea unei cereri. Importați `time` și măsurați cât durează o cerere către pagina principală, apelând `time.perf_counter()` înainte și după. Comparați rezultatul cu `response.elapsed`.

Exercițiul 8. Când un import eșuează. Scrieți un script care încearcă `import bs4` într-un bloc `try` / `except ImportError`. Dacă modulul lipsește, afișați: `Instalați modulul cu: pip install beautifulsoup4`.



```python
# Laborator: funcții, metode și importuri pe web
# Student: <Morozovschii Carolina>

import time

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

# Exercițiul 1. Primul import
import requests
print("Versiune requests:", requests.__version__)

import urllib.request

# Exercițiul 2. Două stiluri de import
res1 = requests.get(BASE_URL, timeout=TIMEOUT)
print("Ex2 - Stil 1 status:", res1.status_code)

from requests import get
res2 = get(BASE_URL, timeout=TIMEOUT)
print("Ex2 - Stil 2 status:", res2.status_code)


# Exercițiul 3. Alias-uri
import requests as rq
res3 = rq.get(BASE_URL, timeout=TIMEOUT)
print("Ex3 - Status cu alias:", res3.status_code)

# Exercițiul 4. Doar biblioteca standard
req = urllib.request.Request(BASE_URL, headers={"User-Agent": "Mozilla/5.0"})
with urllib.request.urlopen(req, timeout=TIMEOUT) as response:
    print("Ex4 - urllib status:", response.status)
    body_bytes = response.read()
    body_text = body_bytes.decode("utf-8")
    print("Ex4 - Primele 200 caractere:", body_text[:200])

# Exercițiul 5. Priviți în interiorul unui modul
print("Ex5 - Dir requests:", dir(requests)[:10])


# Exercițiul 6. Citiți documentația

res_timeout = requests.get(BASE_URL, timeout=5)
print("Ex6 - Cerere cu timeout reușită, status:", res_timeout.status_code)

# Exercițiul 7. Cronometrarea unei cereri
t_start = time.perf_counter()
res_time = requests.get(BASE_URL, timeout=TIMEOUT)
t_end = time.perf_counter()
duration_perf = t_end - t_start
duration_elapsed = res_time.elapsed.total_seconds()
print(f"Ex7 - Măsurat cu perf_counter: {duration_perf:.4f}s | time.elapsed: {duration_elapsed:.4f}s")

# Exercițiul 8. Când un import eșuează
try:
    import bs4
    print("Ex8 - BeautifulSoup4 este instalat!")
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```

Fig.1.1- Codul sursă (part1_imports.py)



```
Problems  Output  Debug Console  Terminal  Ports                                                    Python

PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code> & C:\Users\moroz\AppData\Local\Programs\Python\Python315\python.exe c:/Users/moroz/OneDrive/Documents/web_lab/part1_i
mports.py
Versiune requests: 2.34.2
Ex2 - Stil 1 status: 200
Ex2 - Stil 2 status: 200
Ex3 - Status cu alias: 200
Ex4 - urllib status: 200
Ex4 - Primele 200 caractere: <!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- TODO: Set the document title to the na
Ex5 - Dir requests: ['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError', 'JSONDecodeError', 'NullHandler', 'PreparedRequest', 'ReadTimeout'
, 'Request']
Ex6 - Cerere cu timeout reușită, status: 200
Ex7 - Măsurat cu perf_counter: 0.5127s | time.elapsed: 0.5116s
Ex8 - BeautifulSoup4 este instalat!
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code>
```

Fig.1.2- Rezultatul codului

Capitolul 2: Metode și atribute ale obiectelor web (exercițiile 9–20)

Scop: învățați să folosiți instrumentele pe care obiectele le au deja. Un **atribut** este o valoare (`response.status_code`); o **metodă** este o acțiune pe care o apelați cu paranteze (`response.json()`). Fișier: `part2_methods.py`.

Exercițiul 9. Elementele de bază ale răspunsului. Descărcați pagina principală și afișați `response.status_code`, `response.ok`, `response.url` și `response.encoding`. Lângă fiecare linie, scrieți un comentariu care spune dacă este un atribut sau o metodă.

Exercițiul 10. Tratarea erorilor cu o metodă. Apelați `response.raise_for_status()` pe pagina principală. Apoi trimiteți o cerere către `BASE_URL + "/this-page-does-not-exist"` și prindeți excepția `requests.HTTPError`. Afișați un mesaj prietenos în loc să lăsați programul să se oprească cu eroare.

Exercițiul 11. Toate antetele. Parcurgeți `response.headers.items()` și afișați fiecare antet în forma `Nume: valoare`.

Exercițiul 12. Metode de dicționar. Folosiți `response.headers.get()` pentru a afișa antetele `Server` și `Content-Type`, cu `"lipsește"` ca valoare implicită. Apoi încercați `response.headers.get("content-type")`, cu litere mici. Ce observați?

Exercițiul 13. Înlănțuirea metodelor de șir. Numărați de câte ori apare cuvântul `cyber` pe pagina principală, cu `response.text.lower().count("cyber")`. Într-un comentariu, explicați de ce puteți înlănțui `.lower()` și `.count()`.

Exercițiul 14. Găsirea titlului. Folosiți `.find()` și felierea (slicing) pe `response.text` pentru a extrage textul dintre `<title>` și `</title>`. Curățați-l cu `.strip()` și afișați-l.

> **Indiciu:** `start = html.find("<title>") + len("<title>")`

Exercițiul 15. Numărarea liniilor. Împărțiți codul HTML în linii cu `.splitlines()`. Afișați câte linii sunt și lungimea celei mai lungi linii, folosind `max(lines, key=len)`.

Exercițiul 16. Verificarea HTTPS. Folosiți `.startswith()` pentru a verifica dacă `response.url` începe cu `https://`. Afișați `Conexiune securizată` sau `Conexiune nesecurizată`.

Exercițiul 17. Urmărirea redirecționărilor. Trimiteți o cerere către `http://cybercor.org` (HTTP simplu, fără S). Parcurgeți `response.history` și afișați codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

Exercițiul 18. HEAD versus GET. Trimiteți o cerere către pagina principală cu `requests.head()` și una cu `requests.get()`. Comparați `len(response.content)` pentru ambele și explicați diferența într-un comentariu.

Exercițiul 19. Cookie-uri. Parcurgeți `response.cookies` și afișați, pentru fiecare cookie, atributele `.name` și `.secure`. Dacă nu există cookie-uri, afișați `Niciun cookie setat`.

Exercițiul 20. Sesiuni. Creați un obiect `requests.Session()` și setați un `User-Agent` personalizat cu `session.headers.update({"User-Agent": "WebLab-<numele vostru>"})`. Trimiteți o cerere către `ECHO_URL + "/headers"` și confirmați în rezultat că antetul a fost trimis.



```python
# Laborator: funcții, metode și importuri pe web
# Student: <Morozovschii Carolina>

import time
import requests

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

# Exercițiul 9. Elementele de bază ale răspunsului
res = requests.get(BASE_URL, timeout=TIMEOUT)
print("status_code:", res.status_code)
print("ok:", res.ok)
print("url:", res.url)
print("encoding:", res.encoding)
# Exercițiul 10. Tratarea erorilor cu o metodă
res.raise_for_status()
time.sleep(1)

try:
    bad_res = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
    bad_res.raise_for_status()
except requests.HTTPError as e:
    print(f"Ex10 - Eroare HTTP prinsă cu succes: {e}")

# Exercițiul 11. Toate antetele
print("--- Antete primite ---")
for key, value in res.headers.items():
    print(f"{key}: {value}")

# Exercițiul 12. Metode de dicționar
server = res.headers.get("Server", "lipsește")
content_type = res.headers.get("Content-Type", "lipsește")
content_type_lower = res.headers.get("content-type", "lipsește")
print(f"Ex12 - Server: {server} | Content-Type: {content_type} | lower: {content_type_lower}")

# Exercițiul 13. Înlănțuirea metodelor de șir
count_cyber = res.text.lower().count("cyber")
print(f"Ex13 - Cuvântul 'cyber' apare de {count_cyber} ori.")

# Exercițiul 14. Găsirea titlului
html_text = res.text
start_idx = html_text.find("<title>") + len("<title>")
end_idx = html_text.find("</title>")
if start_idx != -1 and end_idx != -1:
    title = html_text[start_idx:end_idx].strip()
    print("Ex14 - Titlu pagină:", title)

# Exercițiul 15. Numărarea liniilor
lines = html_text.splitlines()
print(f"Ex15 - Număr linii: {len(lines)}")
if lines:
    longest_line = max(lines, key=len)
    print(f"Ex15 - Lungimea celei mai lungi linii: {len(longest_line)}")

# Exercițiul 16. Verificarea HTTPS
if res.url.startswith("https://"):
    print("Ex16 - Conexiune securizată")
else:
    print("Ex16 - Conexiune nesecurizată")

# Exercițiul 17. Urmărirea redirecționărilor
time.sleep(1)
res_redir = requests.get("http://cybercor.org", timeout=TIMEOUT)
print("Ex17 - Istoric redirecționări:")
for r in res_redir.history:
    print(f"  {r.status_code} -> {r.url}")
print("URL final:", res_redir.url)

# Exercițiul 18. HEAD versus GET
time.sleep(1)
res_head = requests.head(BASE_URL, timeout=TIMEOUT)
res_get = requests.get(BASE_URL, timeout=TIMEOUT)
print(f"Ex18 - Lungime corp HEAD: {len(res_head.content)} | GET: {len(res_get.content)}")


# Exercițiul 19. Cookie-uri
if res_get.cookies:
    for cookie in res_get.cookies:
        print(f"Cookie: {cookie.name} | Secure: {cookie.secure}")
else:
    print("Ex19 - Niciun cookie setat")

# Exercițiul 20. Sesiuni
session = requests.Session()
session.headers.update({"User-Agent": "WebLab-Student"})
time.sleep(1)
res_sess = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
data = res_sess.json()
print("Ex20 - Antete confirmate de server:", data.get("headers", {}).get("User-Agent"))
```

Fig.2.1- Codul sursă (part2_methods.py)

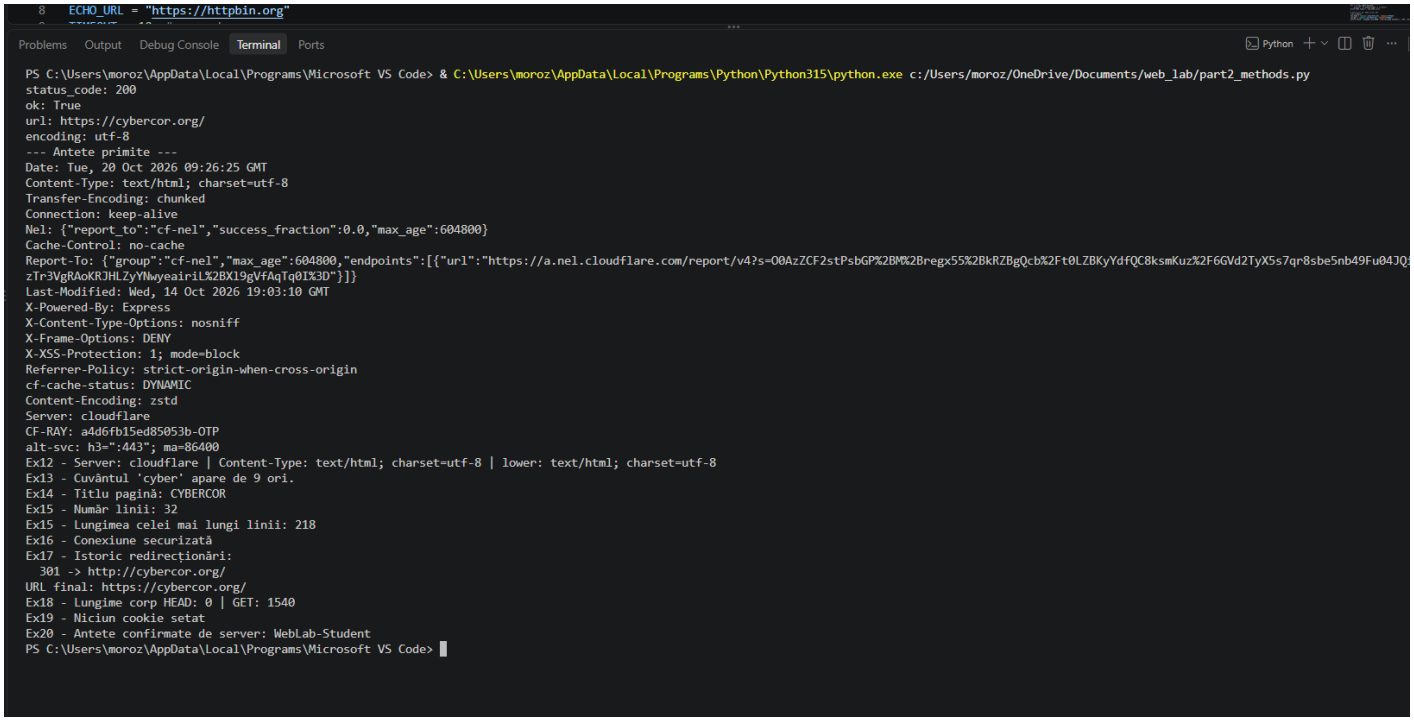


Fig.2.2 – Rezultatul codului

Partea 3: Scrierea propriilor funcții (exercițiile 21–34)

Scop: organizați codul în funcții reutilizabile. De acum înainte, fiecare exercițiu este o funcție pe care o apelați cel puțin o dată. Fișier: `part3_functions.py`.

```
def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code
```

Exercițiul 21. Prima funcție. Scrieți `fetch(url)`, care returnează obiectul răspuns. Apelați-o pentru `BASE_URL` și afișați codul de stare.

Exercițiul 22. Returnarea unei singure valori. Scrieți `get_status(url)`, care returnează doar codul de stare. Apelați-o pentru `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

Exercițiul 23. Parametri impliciți. Modificați funcția în `fetch(url, timeout=10)`. Apelați-o o dată cu valoarea implicită și o dată cu `timeout=3`.

Exercițiul 24. O singură sarcină pe funcție. Scrieți `get_title(html)`, care primește un șir HTML și returnează titlul paginii, refolosind logica din exercițiul 14. Observați că această funcție nu face nicio operație de rețea: descărcarea și analiza sunt acum separate.

Exercițiul 25. Docstring-uri. Adăugați un docstring funcției `get_title` și afișați-l cu `help(get_title)`.

Exercițiul 26. Adnotări de tip. Adăugați adnotări de tip (type hints) funcțiilor, de exemplu `def get_status(url: str) -> int:`. Apelați `get_status(123)` și observați ce se întâmplă. Vă opresc adnotările de tip?

Exercițiul 27. Fără opriri neașteptate. Scrieți `page_exists(url)`, care returnează `True` sau `False`. Folosiți `try` / `except requests.RequestException`, astfel încât funcția să returneze `False` în loc să se oprească cu eroare, chiar și pentru un URL precum `https://this-domain-does-not-exist.invalid`.

Exercițiul 28. Returnarea unui dicționar. Scrieți `check_paths(base, paths)`, care primește o listă precum `["/", "/robots.txt", "/sitemap.xml"]` și returnează un dicționar `{path: status_code}`.

Exercițiul 29. Argumente cu nume. Scrieți `get_header(url, name, default="lipsește")`, care returnează valoarea unui singur antet. Apelați-o cu argumente cu nume: `get_header(BASE_URL, name="Server")`.

Exercițiul 30. Antete de securitate. Scrieți `security_headers(url)`, care verifică prezența acestor cinci antete și returnează `{antet: True/False}`:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Referrer-Policy`

Exercițiul 31. Funcții care apelează funcții. Scrieți `score_headers(results)`, care primește dicționarul din exercițiul 30 și returnează un șir precum `"3/5"`. Apelați-le împreună: `score_headers(security_headers(BASE_URL))`.

Exercițiul 32. Două funcții, o sarcină. Scrieți `fetch_robots(base)`, care returnează textul fișierului `/robots.txt` sau `None` dacă fișierul lipsește. Apoi scrieți `disallowed_paths(robots_text)`, care returnează o listă cu toate valorile `Disallow:`. Asigurați-vă că funcționează și atunci când primește `None`.

Exercițiul 33. Oricâte argumente. Scrieți `response_times(*urls)`, care returnează un dicționar `{url: secunde}`. Apelați-o cu două URL-uri.

Exercițiul 34. Argumente cu nume colectate. Scrieți `log(message, **details)`, care afișează mesajul urmat de fiecare detaliu. Exemplu: `log("verificat", url=BASE_URL, status=200)` afișează `verificat | url=https://cybercor.org | status=200`.



```
C: > Users > moroz > OneDrive > Documents > web_lab > part3_functions.py > ...
# Laborator: funcții, metode și importuri pe web
# Student: <Morozovschii Carolina>

import time
import requests
from typing import Dict, List, Optional

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

# Exercițiul 21. Prima funcție
def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Efectuează o cerere GET către URL-ul specificat și returnează obiectul răspuns."""
    return requests.get(url, timeout=timeout)

print("Ex21 - Status fetch:", fetch(BASE_URL).status_code)

# Exercițiul 22. Returnarea unei singure valori
def get_status(url: str) -> int:
    """Returnează codul de stare HTTP pentru adresa url."""
    res = fetch(url)
    return res.status_code

for path in ["/", "/robots.txt", "/sitemap.xml"]:
    print(f"Ex22 - Status {path}:", get_status(BASE_URL + path))
    time.sleep(1)

# Exercițiul 23. Parametri impliciți
print("Ex23 - Implicit timeout:", fetch(BASE_URL).status_code)
print("Ex23 - Custom timeout:", fetch(BASE_URL, timeout=3).status_code)

# Exercițiul 24. O singură sarcină pe funcție & Exercițiul 25. Docstring-uri & Exercițiul 26. Adnotări de tip
def get_title(html: str) -> str:
    """Extrage și curăță titlul dintr-un cod sursă HTML.

    Primește un string HTML și returnează textul din tagul <title>.
    """
    start = html.find("<title>")
    end = html.find("</title>")
    if start != -1 and end != -1:
        return html[start + len("<title>"):end].strip()
    return "Fără titlu"

print("Ex25 - Help get_title:")
help(get_title)

# Exercițiul 27. Fără opriri neașteptate
def page_exists(url: str) -> bool:
    """Verifică dacă o pagină există fără a opri programul la erori de rețea."""
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return res.status_code < 400
    except requests.RequestException:
        return False

print("Ex27 - Există domeniul invalid?:", page_exists("https://this-domain-does-not-exist.invalid"))

# Exercițiul 28. Returnarea unui dicționar
def check_paths(base: str, paths: List[str]) -> Dict[str, int]:
    """Returnează un dicționar cu codurile de stare pentru fiecare cale."""
    results = {}
    for p in paths:
        full_url = base.rstrip("/") + "/" + p.lstrip("/")
        try:
            res = requests.get(full_url, timeout=TIMEOUT)
            results[p] = res.status_code
        except requests.RequestException:
            results[p] = 0
        time.sleep(1)
    return results

print("Ex28 - Check paths:", check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))

# Exercițiul 29. Argumente cu nume
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Returnează valoarea unui antet specific dintr-un URL."""
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return res.headers.get(name, default)
    except requests.RequestException:
        return default

print("Ex29 - Server header:", get_header(BASE_URL, name="Server"))

# Exercițiul 30. Antete de securitate
def security_headers(url: str) -> Dict[str, bool]:
    """Verifică prezența celor 5 antete esențiale de securitate."""
    sec_list = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return {h: h in res.headers for h in sec_list}
    except requests.RequestException:
        return {h: False for h in sec_list}

print("Ex30 - Sec headers:", security_headers(BASE_URL))
```

Fig.3.1-Codul sursă (part3_functions.py)



```python
# Exercițiul 29. Argumente cu nume
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Returnează valoarea unui antet specific dintr-un URL."""
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return res.headers.get(name, default)
    except requests.RequestException:
        return default

print("Ex29 - Server header:", get_header(BASE_URL, name="Server"))

# Exercițiul 30. Antete de securitate
def security_headers(url: str) -> Dict[str, bool]:
    """Verifică prezența celor 5 antete esențiale de securitate."""
    sec_list = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return {h: h in res.headers for h in sec_list}
    except requests.RequestException:
        return {h: False for h in sec_list}

print("Ex30 - Sec headers:", security_headers(BASE_URL))

# Exercițiul 31. Funcții care apelează funcții
def score_headers(results: Dict[str, bool]) -> str:
    """Returnează un scor sub formă de șir 'X/Y' bazat pe antetele prezente."""
    present = sum(1 for val in results.values() if val)
    total = len(results)
    return f"{present}/{total}"

print("Ex31 - Scor antete:", score_headers(security_headers(BASE_URL)))

# Exercițiul 32. Două funcții, o sarcină
def fetch_robots(base: str) -> Optional[str]:
    """Descarcă fișierul robots.txt sau returnează None dacă lipsește/eșuează."""
    try:
        res = requests.get(base.rstrip("/") + "/robots.txt", timeout=TIMEOUT)
        if res.status_code == 200:
            return res.text
        return None
    except requests.RequestException:
        return None

def disallowed_paths(robots_text: Optional[str]) -> List[str]:
    """Extrage căile interzise (Disallow) din textul robots.txt."""
    if not robots_text:
        return []
    paths = []
    for line in robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            parts = line.split(":", 1)
            if len(parts) > 1:
                val = parts[1].strip()
                if val:
                    paths.append(val)
    return paths

robots = fetch_robots(BASE_URL)
print("Ex32 - Căi interzise:", disallowed_paths(robots))

# Exercițiul 33. Oricâte argumente
def response_times(*urls: str) -> Dict[str, float]:
    """Măsoară timpul de răspuns pentru oricâte URL-uri primite ca argumente."""
    times = {}
    for url in urls:
        try:
            t0 = time.perf_counter()
            requests.get(url, timeout=TIMEOUT)
            t1 = time.perf_counter()
            times[url] = round(t1 - t0, 4)
        except requests.RequestException:
            times[url] = -1.0
        time.sleep(1)
    return times

print("Ex33 - Timpi răspuns:", response_times(BASE_URL, ECHO_URL))

# Exercițiul 34. Argumente cu nume colectate
def log(message: str, **details) -> None:
    """Afișează un mesaj urmat de perechile cheie=valoare din kwargs."""
    details_str = " | ".join(f"{k}={v}" for k, v in details.items())
    if details_str:
        print(f"{message} | {details_str}")
    else:
        print(message)

log("verificat", url=BASE_URL, status=200)
```

Fig.3.2- Codul sursă (part3_functions.py)

```
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code> & C:\Users\moroz\AppData\Local\Programs\Python\Python315\python.exe c:/Users/moroz/OneDrive/Documents/web_lab/part3_functions.py
Ex21 - Status fetch: 200
Ex22 - Status /: 200
Ex22 - Status /robots.txt: 200
Ex22 - Status /sitemap.xml: 200
Ex23 - Implicit timeout: 200
Ex23 - Custom timeout: 200
Ex25 - Help get_title:
Help on function get_title in module __main__:

get_title(html: str) -> str
    Extrage \u0219i cur\u0103\u021b\u0103 titlul dintr-un cod surs\u0103 HTML.

    Primeste un string HTML \u0219i returneaz\u0103 textul din tagul <title>.

Ex27 - Există domeniul invalid?: False
Ex28 - Check paths: {'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
Ex29 - Server header: cloudflare
Ex30 - Sec headers: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}
Ex31 - Scor antete: 3/5
Ex32 - Căi interzise: []
Ex33 - Timpi răspuns: {'https://cybercor.org': 0.4088, 'https://httpbin.org': 1.0521}
verificat | url=https://cybercor.org | status=200
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code>
```

Fig.3.3- Rezultatul codului

Capitol 4: Alte module din biblioteca standard (exercițiile 35–43)

Scop: combinați mai multe importuri pentru a rezolva probleme mai mari. Puneți fiecare soluție într-o funcție. Fișier: `part4_modules.py`.

Modul	La ce îl folosiți aici
`urllib.parse`	Descompunerea și compunerea URL-urilor
`re`	Căutarea de tipare în text
`html.parser`	Citirea corectă a codului HTML
`hashlib`	Calcularea amprentei unei pagini
`json`	Salvarea datelor într-un fișier
`socket`	Aflarea adreselor IP și deschiderea conexiunilor
`ssl` + `datetime`	Citirea datei de expirare a unui certificat

Exercițiul 35. Descompunerea unui URL. Folosiți `urllib.parse.urlparse()` pe `https://cybercor.org/path?x=1#top`. Afișați `.scheme`, `.netloc`, `.path`, `.query` și `.fragment`.

Exercițiul 36. Compunerea URL-urilor. Folosiți `urllib.parse.urljoin()` pentru a transforma legăturile relative `/about`, `contact.html` și `../index.html` în URL-uri complete, pornind de la `BASE_URL`.

Exercițiul 37. Extragerea legăturilor. Folosiți `re.findall(r'href="([^"]+)"', html)` pentru a obține toate legăturile de pe pagina principală. Puneți codul într-o funcție `extract_links(html)`, care returnează legăturile fără duplicate.

Exercițiul 38. Interne sau externe? Scrieți `split_links(links, domain)`, care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri. Folosiți mai întâi `urljoin()`, apoi `urlparse().netloc`.

Exercițiul 39. Propriile metode. Creați o clasă care moștenește `html.parser.HTMLParser` și suprascrieți metoda `handle_starttag()`, pentru a colecta atributul `src` al fiecărui tag `<img>`.

```
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # codul vostru aici
        pass
```

> **Indiciu:** `attrs` este o listă de perechi `(nume, valoare)`, de exemplu `[("src", "/logo.png"), ("alt", "Logo")]`. Expresia `dict(attrs).get("src")` vă dă valoarea atributului `src` sau `None`.

Cum testați exercițiul 39. Testați mai întâi parserul pe un fragment HTML scris de voi, pentru care știți dinainte rezultatul corect. Abia după aceea rulați-l pe site-ul real. Adăugați acest test sub clasa voastră:

```
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                          # imagine obișnuită
    "poza.jpg",                           # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Dacă testul trece, parserul vostru: găsește tag-urile `<img>` indiferent de majuscule, ignoră tag-urile `<img>` fără `src`, ignoră celelalte tag-uri (de exemplu `<a>`) și păstrează ordinea din pagină. Dacă `assert` eșuează, comparați lista afișată de `print()` cu lista așteptată.

Apoi rulați parserul pe pagina reală:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Verificați rezultatul în două moduri: comparați numărul găsit cu `response.text.lower().count("<img")` și deschideți pagina în browser, apoi apăsați Ctrl+U ca să vedeți codul sursă. O listă goală nu înseamnă neapărat o greșeală: unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`. Tocmai de aceea testul cu HTML cunoscut vine primul.

Exercițiul 40. Amprenta paginii. Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

Exercițiul 41. Salvarea în JSON. Salvați antetele paginii principale în `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Apoi încărcați fișierul înapoi cu `json.load()` și afișați un antet din el.

Exercițiul 42. Interogare DNS. Scrieți `resolve(hostname)`, care returnează `socket.gethostbyname(hostname)`. Afișați adresa IP a domeniului `cybercor.org`.

Exercițiul 43. Expirarea certificatului. Scrieți `cert_days_left(hostname)`, care se conectează la portul 443, citește certificatul și returnează câte zile mai sunt până la expirarea lui.

> **Indiciu:** folosiți `ssl.create_default_context()`, `socket.create_connection((hostname, 443))`, `context.wrap_socket(sock, server_hostname=hostname)` și `getpeercert()["notAfter"]`. Convertiți data cu `ssl.cert_time_to_seconds()`.

C: > Users > moroz > OneDrive > Documents > web_lab > part4_modules.py > ...

```python
# Laborator: funcții, metode și importuri pe web
# Student: <Morozovschii Carolina>

import time
import requests
import urllib.parse
import re
from html.parser import HTMLParser
import hashlib
import json
import socket
import ssl
import datetime

BASE_URL = "https://cybercor.org"
TIMEOUT = 10

# Exercițiul 35. Descompunerea unui URL
parsed = urllib.parse.urlparse("https://cybercor.org/path?x=1#top")
print(f"Ex35 - scheme: {parsed.scheme}, netloc: {parsed.netloc}, path: {parsed.path}, query: {parsed.query}, fragment: {parsed.fragment}")

# Exercițiul 36. Compunerea URL-urilor
for rel in ["/about", "contact.html", "../index.html"]:
    full = urllib.parse.urljoin(BASE_URL, rel)
    print(f"Ex36 - Relative: {rel} -> Absolute: {full}")

# Exercițiul 37. Extragerea legăturilor
def extract_links(html: str) -> list:
    """Extrage toate legăturile href din HTML fără duplicate."""
    raw_links = re.findall(r'href="([^"]+)"', html, re.IGNORECASE)
    return list(set(raw_links))

res_main = requests.get(BASE_URL, timeout=TIMEOUT)
links = extract_links(res_main.text)
print(f"Ex37 - Legături găsite ({len(links)}):", links[:5])

# Exercițiul 38. Interne sau externe?
def split_links(links: list, domain: str) -> tuple:
    """Separă legăturile în interne și externe."""
    internal = []
    external = []
    base_netloc = urllib.parse.urlparse(domain).netloc.lower()

    for l in links:
        full_url = urllib.parse.urljoin(domain, l)
        netloc = urllib.parse.urlparse(full_url).netloc.lower()
        if netloc == base_netloc or not netloc:
            internal.append(full_url)
        else:
            external.append(full_url)
    return list(set(internal)), list(set(external))

int_links, ext_links = split_links(links, BASE_URL)
print(f"Ex38 - Interne: {len(int_links)} | Externe: {len(ext_links)}")

# Exercițiul 39. Propriile metode
class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag.lower() == "img":
            attrs_dict = dict(attrs)
```

Fig.4.1-Codul sursă (part4_modules.py)



Fig.4.2-Codul sursă(part4_modules.py)

```python

finder = ImageFinder()
finder.feed(test_html)
print("Ex39 - Imagini test găsite:", finder.images)

assert finder.images == [
    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul Ex39 a trecut!")

# Rulare pe pagina reală
finder_real = ImageFinder()
finder_real.feed(res_main.text)
print(f"Ex39 - Real: {len(finder_real.images)} imagini găsite pe {BASE_URL}")

# Exercițiul 40. Amprenta paginii
def page_fingerprint(url: str) -> str:
    """Returnează hash-ul SHA-256 al conținutului paginii."""
    r = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(r.content).hexdigest()

fp1 = page_fingerprint(BASE_URL)
time.sleep(1)
fp2 = page_fingerprint(BASE_URL)
print("Ex40 - FP1:", fp1)
print("Ex40 - FP2:", fp2)

# Exercițiul 41. Salvarea în JSON
with open("headers.json", "w", encoding="utf-8") as f:
    json.dump(dict(res_main.headers), f, indent=2)

with open("headers.json", "r", encoding="utf-8") as f:
    loaded_headers = json.load(f)
print("Ex41 - Antet reîncărcat:", loaded_headers.get("Content-Type"))

# Exercițiul 42. Interogare DNS
def resolve(hostname: str) -> str:
    """Returnează adresa IP a host-ului."""
    return socket.gethostbyname(hostname)

host = urllib.parse.urlparse(BASE_URL).netloc
print(f"Ex42 - Adresă IP pentru {host}:", resolve(host))

# Exercițiul 43. Expirarea certificatului
def cert_days_left(hostname: str) -> int:
    """Returnează numărul de zile rămase până la expirarea certificatului SSL."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()
            not_after_str = cert['notAfter']
            expire_date = datetime.datetime.strptime(not_after_str, "%b %d %H:%M:%S %Y %Z")
            now = datetime.datetime.utcnow()
            days = (expire_date - now).days
            return days

print("Ex43 - Zile certificat rămase:", cert_days_left(host))
```

Fig.4.3-Codul sursă(part4_modules.py)

```
ybercor.org', port=443): Failed to resolve 'cybercor.org' ([Errno 11001] getaddrinfo failed))
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code> & C:\Users\moroz\AppData\Local\Programs\Python\Python315\python.exe c:/Users/moroz/OneDrive/Documents/web_lab/part4_m
odules.py
Ex35 - scheme: https, netloc: cybercor.org, path: /path, query: x=1, fragment: top
Ex36 - Relative: /about -> Absolute: https://cybercor.org/about
Ex36 - Relative: contact.html -> Absolute: https://cybercor.org/contact.html
Ex36 - Relative: ../index.html -> Absolute: https://cybercor.org/index.html
Ex37 - Legături găsite (1): ['/assets/index-CErP0EKt.css']
Ex38 - Interne: 1 | Externe: 0
Ex39 - Imagini test găsite: ['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul Ex39 a trecut!
Ex39 - Real: 0 imagini găsite pe https://cybercor.org
Ex40 - FP1: 7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
Ex40 - FP2: 7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
Ex41 - Antet reîncărcat: text/html; charset=utf-8
Ex42 - Adresă IP pentru cybercor.org: 104.21.91.71
c:\Users\moroz\OneDrive\Documents\web_lab\part4_modules.py:133: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use t
imezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow()
Ex43 - Zile certificat rămase: 44
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code>
```

Fig.4.4-Rezultatul codului

Capitol 5: Construirea propriului modul (exercițiile 44–50)

Scop: transformați funcțiile într-un modul pe care alte programe îl pot importa, la fel ca `requests`. La final, directorul vostru ar trebui să arate astfel:

```
web_lab/
├── webtools.py     # modulul vostru: toate funcțiile reutilizabile
├── main.py         # programul care importă webtools
├── report.csv      # creat în exercițiul 49
└── report.json     # creat în exercițiul 50
```

Exercițiul 44. Crearea unui modul. Mutați `fetch`, `get_status`, `get_title` și `security_headers` într-un fișier nou, `webtools.py`. În `main.py`, scrieți `import webtools` și apelați `webtools.get_title(webtools.fetch(BASE_URL).text)`.

Exercițiul 45. Importarea anumitor nume. În `main.py`, treceți la `from webtools import get_title, security_headers`. Într-un comentariu, explicați ce s-ar întâmpla dacă `main.py` ar avea și o funcție proprie numită `get_title`.

Exercițiul 46. Blocul main. Adăugați acest bloc la finalul fișierului `webtools.py`:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rulați `python webtools.py`, apoi `python main.py`. Explicați de ce autotestul rulează doar în primul caz.

Exercițiul 47. Constante de modul. Adăugați constanta `DEFAULT_HEADERS = {"User-Agent": "WebLab-<numele vostru>"}` în `webtools.py` și folosiți-o în `fetch()`. În `main.py`, importați constanta și afișați-o.

Exercițiul 48. Argumente din linia de comandă. Folosiți `argparse` în `main.py`, astfel încât programul să ruleze ca `python main.py https://cybercor.org`. Transmiteți argumentul URL funcțiilor voastre, în loc să folosiți `BASE_URL`.

Exercițiul 49. Un raport CSV. Folosiți modulul `csv` pentru a salva rezultatul funcției `check_paths()` în `report.csv`, cu coloanele `path`, `status` și `checked_at`. Pentru oră, folosiți `datetime.now().isoformat()`.

Exercițiul 50. Proiect final: raportul site-ului. Scrieți `site_report(url)` în `webtools.py`. Funcția trebuie să folosească funcțiile voastre pentru a colecta toate informațiile de mai jos, să le afișeze și să le salveze în `report.json`:

1. Codul de stare și URL-ul final
2. Titlul paginii
3. Adresa IP
4. Lanțul de redirecționări pornind de la `http://`
5. Scorul antetelor de securitate, de exemplu `3/5`
6. Zilele rămase până la expirarea certificatului
7. Numărul de legături interne și externe
8. Căile interzise din `robots.txt`

Un exemplu de rezultat (valorile voastre vor fi diferite):

```
=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         203.0.113.10
Redirecționări:    301 -> https://cybercor.org/
Scor securitate:   3/5
Certificat:        74 de zile rămase
Legături:          38 interne, 12 externe
Căi interzise:     /admin, /private
Salvat în report.json
```

> **Bonus:** instalați `beautifulsoup4` și rescrieți `get_title()` și `extract_links()` folosind această bibliotecă. Comparați cât de mult mai scurte și mai sigure devin.

```python
# webtools.py - Modul reutilizabil
# Laborator: funcții, metode și importuri pe web
# Student: <Morozovschii Carolina>

import time
import requests
import urllib.parse
import re
import socket
import ssl
import datetime
from typing import Dict, List, Optional, Tuple

DEFAULT_HEADERS = {"User-Agent": "WebLab-Student"}
TIMEOUT = 10

def fetch(url: str, headers: dict = DEFAULT_HEADERS, timeout: int = TIMEOUT) -> requests.Response:
    """Trimite o cerere GET către un URL specificat."""
    return requests.get(url, headers=headers, timeout=timeout)

def get_status(url: str) -> int:
    """Returnează codul de stare HTTP pentru adresa url."""
    res = fetch(url)
    return res.status_code

def get_title(html: str) -> str:
    """Extrage titlul paginii dintr-un șir HTML."""
    start = html.find("<title>")
    end = html.find("</title>")
    if start != -1 and end != -1:
        return html[start + len("<title>"):end].strip()
    return "Fără titlu"

def security_headers(url: str) -> Dict[str, bool]:
    """Verifică prezența celor 5 antete esențiale de securitate."""
    sec_list = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    try:
        res = fetch(url)
        return {h: h in res.headers for h in sec_list}
    except requests.RequestException:
        return {h: False for h in sec_list}

def score_headers(results: Dict[str, bool]) -> str:
    """Returnează un scor sub formă de șir 'X/Y'."""
    present = sum(1 for val in results.values() if val)
    return f"{present}/{len(results)}"

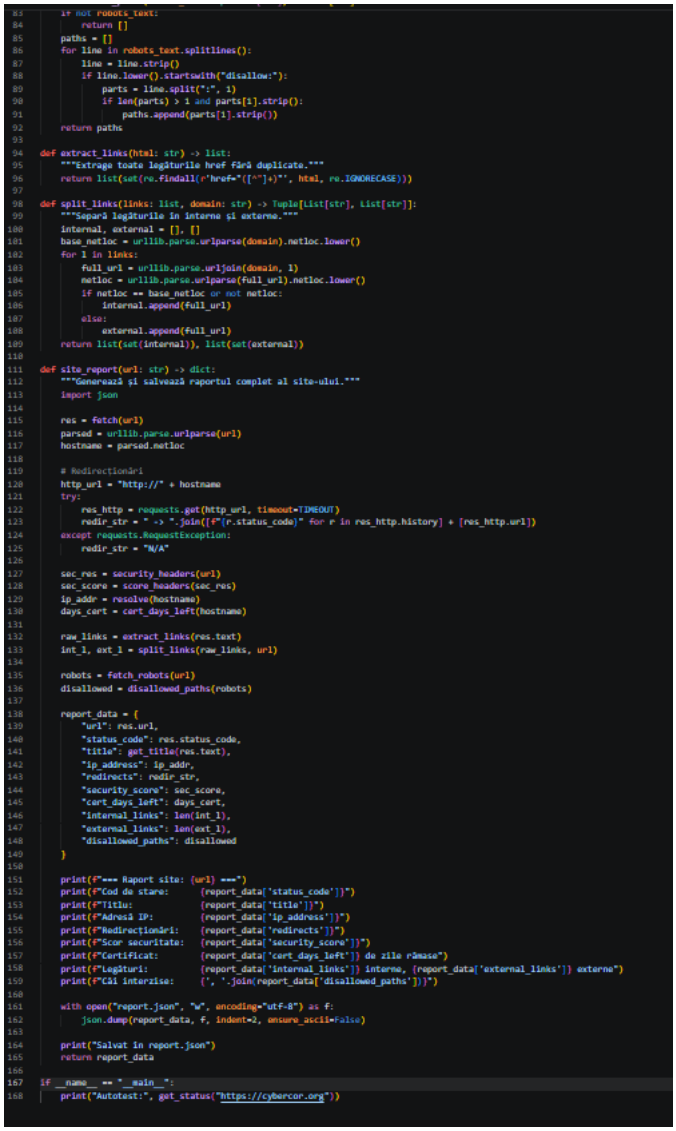
def resolve(hostname: str) -> str:
    """Returnează adresa IP a host-ului."""
    try:
        return socket.gethostbyname(hostname)
    except socket.error:
        return "N/A"

def cert_days_left(hostname: str) -> int:
    """Returnează numărul de zile rămase până la expirarea certificatului SSL."""
    try:
        context = ssl.create_default_context()
        with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
            with context.wrap_socket(sock, server_hostname=hostname) as ssock:
                cert = ssock.getpeercert()
                expire_date = datetime.datetime.strptime(cert['notAfter'], "%b %d %H:%M:%S %Y %Z")
                return (expire_date - datetime.datetime.utcnow()).days
    except Exception:
        return 0

def fetch_robots(base: str) -> Optional[str]:
    """Descarcă textul din robots.txt."""
    try:
        res = fetch(base.rstrip("/") + "/robots.txt")
        return res.text if res.status_code == 200 else None
    except requests.RequestException:
        return None

def disallowed_paths(robots_text: Optional[str]) -> List[str]:
    """Extrage căile Disallow din robots.txt."""
    if not robots_text:
```

Fig.5.1-Codul sursă(webtools.py)



```
    if not robots_text:
        return []
    paths = []
    for line in robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            parts = line.split(":", 1)
            if len(parts) > 1 and parts[1].strip():
                paths.append(parts[1].strip())
    return paths

def extract_links(html: str) -> list:
    """Extrage toate legăturile href fără duplicate."""
    return list(set(re.findall(r'href="([^"]+)"', html, re.IGNORECASE)))

def split_links(links: list, domain: str) -> Tuple[List[str], List[str]]:
    """Separă legăturile în interne și externe."""
    internal, external = [], []
    base_netloc = urllib.parse.urlparse(domain).netloc.lower()
    for l in links:
        full_url = urllib.parse.urljoin(domain, l)
        netloc = urllib.parse.urlparse(full_url).netloc.lower()
        if netloc == base_netloc or not netloc:
            internal.append(full_url)
        else:
            external.append(full_url)
    return list(set(internal)), list(set(external))

def site_report(url: str) -> dict:
    """Generează și salvează raportul complet al site-ului."""
    import json

    res = fetch(url)
    parsed = urllib.parse.urlparse(url)
    hostname = parsed.netloc

    # Redirecționări
    http_url = "http://" + hostname
    try:
        res_http = requests.get(http_url, timeout=TIMEOUT)
        redir_str = " -> ".join([f"{r.status_code}" for r in res_http.history] + [res_http.url])
    except requests.RequestException:
        redir_str = "N/A"

    sec_res = security_headers(url)
    sec_score = score_headers(sec_res)
    ip_addr = resolve(hostname)
    days_cert = cert_days_left(hostname)

    raw_links = extract_links(res.text)
    int_l, ext_l = split_links(raw_links, url)

    robots = fetch_robots(url)
    disallowed = disallowed_paths(robots)

    report_data = {
        "url": res.url,
        "status_code": res.status_code,
        "title": get_title(res.text),
        "ip_address": ip_addr,
        "redirects": redir_str,
        "security_score": sec_score,
        "cert_days_left": days_cert,
        "internal_links": len(int_l),
        "external_links": len(ext_l),
        "disallowed_paths": disallowed
    }

    print(f"=== Raport site: {url} ===")
    print(f"Cod de stare:      {report_data['status_code']}")
    print(f"Titlu:             {report_data['title']}")
    print(f"Adresă IP:         {report_data['ip_address']}")
    print(f"Redirecționări:    {report_data['redirects']}")
    print(f"Scor securitate:   {report_data['security_score']}")
    print(f"Certificat:        {report_data['cert_days_left']} de zile rămase")
    print(f"Legături:          {report_data['internal_links']} interne, {report_data['external_links']} externe")
    print(f"Căi interzise:     {', '.join(report_data['disallowed_paths'])}")

    with open("report.json", "w", encoding="utf-8") as f:
        json.dump(report_data, f, indent=2, ensure_ascii=False)

    print("Salvat în report.json")
    return report_data

if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Fig.5.2- Codul sursă(webtools.py)

```
Ex43 - zile certificat rămase: 44
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code> & C:\Users\moroz\AppData\Local\Programs\Python\Python315\python.exe c:/Users/moroz/OneDrive/Documents/web_lab/webtool
s.py
Autotest: 200
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code>
```

Fig.5.3-Rezultatul codului

```python
# main.py
# Laborator: funcții, metode și importuri pe web
# Student: <Morozovschii Carolina>

import argparse
import csv
import datetime
import webtools
from webtools import get_title, security_headers, DEFAULT_HEADERS

def main():
    print("DEFAULT_HEADERS din modul:", DEFAULT_HEADERS)

    # Exercițiul 48. Argumente din linia de comandă
    parser = argparse.ArgumentP(parameter) nargs: int | str | None b pentru un URL.")
    parser.add_argument("url", nargs="?", default="https://cybercor.org", help="URL-ul de analizat")
    args = parser.parse_args()

    target_url = args.url

    # Exercițiul 49. Un raport CSV
    paths_to_check = ["/", "/robots.txt", "/sitemap.xml"]
    print("Se generează report.csv...")
    with open("report.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["path", "status", "checked_at"])
        for p in paths_to_check:
            full = target_url.rstrip("/") + "/" + p.lstrip("/")
            st = webtools.get_status(full)
            writer.writerow([p, st, datetime.datetime.now().isoformat()])
    print("Salvat în report.csv")

    # Exercițiul 50. Proiect final
    webtools.site_report(target_url)

if __name__ == "__main__":
    main()
```

Fig.5.4-Codul sursă(main.py)

```
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code> & C:\Users\moroz\AppData\Local\Programs\Python\Python315\python.exe c:/Users/moroz/OneDrive/Documents/web_lab/main.py

DEFAULT_HEADERS din modul: {'User-Agent': 'WebLab-Student'}
Se generează report.csv...
Salvat în report.csv
=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         172.67.211.96
Redirecționări:    301 -> https://cybercor.org/
Scor securitate:   3/5
Certificat:        44 de zile rămase
Legături:          1 interne, 0 externe
Căi interzise:
Salvat în report.json
PS C:\Users\moroz\AppData\Local\Programs\Microsoft VS Code>
```

Fig.5.5-Rezultatul codului

CONCLUZII

Laboratorul oferă o pregătire practică în programare web cu Python, învățând lucrul cu protocolul HTTP, utilizarea modulelor (requests, biblioteca standard) și bunele practici de codare (funcții, tratarea erorilor, documentare). La final, toate noțiunile sunt consolidate prin crearea unui modul propriu (webtools.py) și a unui utilitar CLI (main.py) care analizează un site și generează rapoarte automatizate în format CSV și JSON.